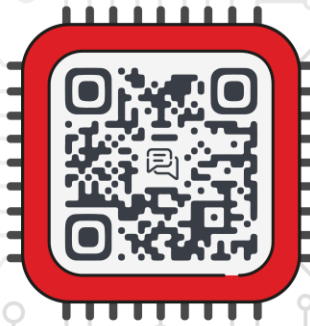




1+2

10

1200 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

مراجعة برمجة 2

د. عملي مشترك



RB Informatics; 23/10/2025

برمجة ٣

- أهلاً بكم زملائنا في بداية فصل جديد ومنتعة جديدة في مادة البرمجة 3 ، سنتعمق في هذا المقرر في لغة البرمجة جافا ونكتشف المزيد من المفاهيم المميزة
- في البداية لتذكر معا ما قد سبق وتعلمنا في مادة البرمجة 2

أساسيات Java :

بداية سنتذكر بعض الأساسيات وكما نعرف فلكتابه أي كود نحتاج لكتابة ما يلي :

```
public class Main {
    public static void main(String[] args) {
    }
}
```

وهي لتنفيذ البرنامج

1. المتغيرات (variables) :

لكتابة أي متغير نحتاج أن نعرف ثلاث أشياء : نوعه _ اسمه _ قيمته .

int x = 5 ;

النوع الاسم القيمة

مثال :

ولدينا عدة أنواع :



- Int للأعداد الصحيحة (-10 ، 5)
- Float للأعداد العشرية مع فواصل (3.14)
- Char للأحرف ' A '
- String للنصوص " hello "
- Boolean للقيم المنطقية (صح أو خطأ) False / True

ملاحظة : لغة java لغة حساسة فمثلا لو كتبنا : `int x = 5 ;`
 ثم كتبنا : `int X = 6 ;`
 فتقرأهم كمتغيرين اثنين لا كواحد .

2. الشروط (if statement) :

الشروط تجعل البرنامج يفكر ويقرر ماذا يفعل حسب الحالة أي إذا تحقق الشرط ← نفذ الكود
 وإذا لم يتحقق ← نفذ كود آخر .

مثال :

```
int x=5 ;
if ( x% 2==0 ) {
    System.out.println("
    العدد زوجي ");
}
else{
    System.out.println("
    العدد فردي ");
}
```

شرح الكود :

الشرط في السطر 2 : إذا تحقق أن باقي قسمة العدد x على 2 تساوي ال 0 فاطبع أن العدد زوجي وإلا اطبع العدد فردي

أنواع تراكيب الشرط :

- If فقط
- If else
- If else if else

3. الحلقات (loops) :

الحلقات تستخدم لتكرار نفس الكود أكثر من مرة بدلا من أن نكتبه يدويا عدة مرات :

1. For loop

نستخدمها عندما نعرف من البداية كم مرة نريد تكرار الكود .

```
for ( int i=0 ; i<3 ; i++){
    System.out.println("Hi");
}
```

نبدأ العداد من الصفر `int i = 0 ;`

ونضع شرط التوقف (استمر طالما `i < 3`) زيادة `i` كل مرة بواحد `i++`

النتيجة : تطبع " Hi " ثلاث مرات .

2. while loop

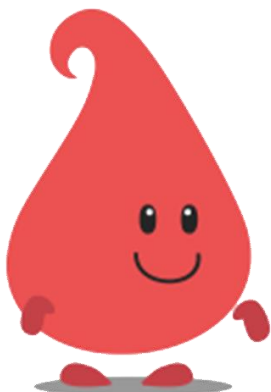
تعمل طالما الشرط صحيح دون أن نعرف كم مرة بالضبط .

```
int i=0 ;
while (i<3) {
    System.out.println("Hi");
    i++;
}
```

3. Do while loop

تشبه ال while loop لكن تنفذ الكود مرة واحدة على الأقل قبل فحص الشرط .

```
int i=0 ;
do{
    System.out.println("Hi");
    i++;
}while(i<3);
```



Java oop

1. الكلاس : class

هو قالب (template) أو مخطط ، يستخدم لإنشاء الكائنات (objects) بالإضافة لتعريف البيانات (Attributes) والسلوكيات (methods) الخاصة بها .

2. Objects :

هو نسخة (instance) من ال class ، يملك الكائن القيم العملية للبيانات ويمكنه تنفيذ ال methods المعرفة داخل ال class .

مثال :

```
public class Car {
    String color ;
    int speed ;
    void drive (){
        System.out.println("the car is
        driving");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Car mycar = new Car();
        // إنشاء كائن سيارة من كلاس
        mycar.color = "Red";
        mycar.speed = 120;
    }
}
```

Class car فيه خصائص ودوال

3. التغليف Encapsulation :

التغليف هي طريقة لنحامي بيانات الكائن من التغيير المباشر ونسمح بالتعامل معها فقط عن طريق دوال خاصة

مثال : تخيل لو لديك خزانة بداخلها أموال لا أحد يستطيع فتحها إلا عن طريق " واجهة " معينة (مثلا كود سري)

ونفس الشيء في البرمجة لا نترك المتغيرات مكشوفة بل نغلفها (نحميها) داخل الكلاس ، نستخدم الكلمة private لحمايتها ودوال الوصول (set , get)

مثال :

```
public class Student {
    private String name ;
    public void setName (String n ) {
        name=n;
        // setter لتغيير القيمة
    }
    public String getName (){
        return name ;
        // getter لعرض القيمة
    }
}
```

ملاحظة : الكلمات المفتاحية (Access Modifiers) هي كلمات تستخدم بتحديد الوصول إلى العناصر من دوال ومتغيرات وكلاسات

الكلمة المفتاحية	مستوى الوصول
Private	خاص جدا الأفضل لحماية البيانات
Default بدون أي كلمة مفتاحية	يتم الوصول فقط داخل الحزمة (package)
Protected	محمي (تستخدم عند الوراثة)
Public	عام (من أي مكان)

4. الوراثة Inheritance

فكرته : لدينا كلاس ابن يرث من كلاس أب أي أنه يأخذ منه الخصائص والدوال .

مثال : الطالب هو نوع من الإنسان ، الإنسان عنده اسم وعمر وطريقة يتكلم فيها و الطالب عنده نفس الأشياء بالإضافة مثلا لرقم جامعي وطريقة الدراسة

نستخدم كلمة extends للوراثة :

مثال :

```
public class Person {
    String name ;
    int age ;
    void speak(){
        System.out.println("I can speak");
    }
}
```

```
public class Student extends Person {
    int id ;
    void study (){
        System.out.println("I'm a student ");
    }
}
```

فائدة الوراثة : إعادة استخدام الكود وتجنب التكرار ، حيث يمكن ل class الابن وراثه بيانات وسلوك الكلاس الأب بالإضافة لتنظيم الكود وتحقيق مفهوم تعددية الأشكال لزيادة المرونة وسهولة التوسع في البرامج .

5. تعددية الأشكال polymorphism :

تعني قدرة ال objects على اتخاذ أشكال متعددة حسب السياق المستخدم فيه ، أي نفس الكود ممكن أن يعمل بعدة طرق حسب السياق ، ببساطة نفس الدالة يمكن أن تتصرف بشكل مختلف حسب الكائن الذي ينفذها .

مثال : عندما تقول " ارسم " الرسام ممكن يرسم دائرة أو مربع أو قلب ، نفس الفعل " ارسم " لكن النتيجة تختلف حسب من ينفذها

1. النوع الأول overloading أو (compile-time polymorphism) :

مثال :

```
public class MathExample {
    int add ( int a , int b ) {
        return a+b ;
    }
    double add ( double a , double b){
        return a+b ;
    }
}
```

يحدث عندما يكون هناك عدة methods بنفس الاسم ولكن بمعاملات مختلفة .

نفس الدالة add لكن تختلف بالأنواع .

2. النوع الثاني override أو (Run-time polymorphism)

يحدث عندما يعيد ال class الفرعي (الابن) تعريف method موجودة في الكلاس الأب بنفس الاسم ونفس المعاملات .

```
public class Animal {
    void sound () {
        System.out.println("Animal sound");
    }
}
```

```
public class Cat extends Animal {
    void sound () {
        System.out.println("meow");
    }
}
```

الدالة نفسها (sound) لكن السلوك تغير حسب الكائن

6. التجريد Abstraction :

الفكرة من التجريد أننا نخفي التفاصيل ونضع المبرمج يركز على ماذا تفعل الدالة وليس كيف تفعلها .

مثال : عندما نستخدم السيارة نضغط زر التشغيل بدون أن يهمنا ماذا يحدث داخل المحرك

نستخدم كلمة abstract لتعريف كلاس فيه دوال بدون تنفيذ ، ولكن الكلاسات التي ترث منه يجب أن تنفذها .

مثال :

```
public abstract class Shape {
    abstract void draw();
}

public class Circle extends Shape {
    void draw(){
        System.out.println("Drawing a circle");
    }
}
```

يعيني نحدد الفكرة العامة

وكل كلاس ابن يطبقها بطريقة الخاصة .

7. الواجهة interface

هو قالب يحدد مجموعة من الدوال تنفذها ، لإجبار ال classes الأبناء على تطبيق هذه الدوال ويحقق مبدأ التجريد الكامل .

```
public interface Drawable {
    void draw();
}

public class Circle implements Drawable {
    public void draw(){
        System.out.println("Drawing circle");
    }
}
```

مثال :

8. العلاقات بين ال classes

1. Association :

وهي علاقة عامة بين كائنين بدون امتلاك مثلاً : طالب وجامعة (الطالب مرتبط بالجامعة لكنها ليست ملكه)

2. Aggregation :

وهي علاقة امتلاك جزئي (الكائن جزء لكن ممكن أن يعيش لوحده)

3. Composition :

علاقة امتلاك قوية (الكائن لا يعيش بدون الآخر) مثل الجسم والقلب .

مشروع java برمجة 2 :

شرح المشروع (فكرته) :

الشخص يذهب للمكتب يستعار كتاب ، المكتبة تسجل اسمه ، ممكن يستعار كتاب _ مشروع ، المكتبة يجب أن تدبر كل هذا

1. Class Book :

كلاس Book يمثل الكتاب الموجود في المكتبة

■ الخصائص (Attributes) :

عنوان الكتاب : title

اسم المؤلف : author

رقم الكتاب : id

■ الدوال (methods) :

دالة لإظهار معلومات الكتاب

+ دوال لتقليل أو زيادة عدد

النسخ عند الاستعارة أو الإرجاع

```

1 public class Book {
2     private String title ;
3     private String author ;
4     private int id ;
5     private int copies ;
6     public Book(String title, String author, int id, int copies) {
7         this.title = title;
8         this.author = author;
9         this.id = id;
10        this.copies = copies;
11    }
12    public String getTitle() {
13        return title;
14    }
15    public void displayInfo(){
16        System.out.println("Book:"+title+"by"+author+"id:"+id);
17    }
18    public boolean isAvailable(){
19        return copies>0;
20    }
21    public void borrowCoby (){
22        if(copies>0)
23            copies--;
24    }
25    public void returnCopy(){
26        copies++;
27    }
28 }
```

2. Class Member :

الكلاس member يمثل عضو المكتبة
مثل
الطالب أو الشخص الذي يستطيع
استعارة كتب .

■ الخصائص :

اسم العضو : name

رقم العضوية : memberId

قائمة الكتب التي استعارها العضو :
Borrowed books

■ الدوال :

دالة لإضافة كتاب إلى قائمة الاستعارة
دالة لإزالة كتاب عند الإرجاع

```

1 import java.util.ArrayList;
2 public class Member {
3     private String name ;
4     private int memberId ;
5     private ArrayList <Book> borrowedBooks ;
6     public Member(String name, int memberId ) {
7         this.name = name;
8         this.memberId = memberId;
9         borrowedBooks = new ArrayList<>();
10    }
11    public void borrowBook(Book book){
12        borrowedBooks.add(book);
13    }
14    public void returnBook(Book book){
15        borrowedBooks.remove(book);
16    }
17    public String getName() {
18        return name;
19    }
20    public ArrayList<Book> getBorrowedBooks() {
21        return borrowedBooks;
22    }
23    }

```

3. Class Library :

الكلاس library يمثل المكتبة نفسها
وهي التي تحتوي على جميع الكتب
والأعضاء
وفيها تتم العمليات الأساسية مثل :
إضافة كتاب ، تسجيل عضو ، الاستعارة ،
الإرجاع .

```

1 import java.util.ArrayList;
2 public class Library {
3     private ArrayList <Book> books;
4     private ArrayList <Member> members;
5     public Library(){
6         books=new ArrayList<>();
7         members=new ArrayList<>();
8     }
9     public void addBook(Book book ) {
10        books.add(book);
11    }
12    public void addMember(Member member){
13        members.add(member);
14    }

```



15 // عملية الاستعارة

```

16 public void borrowBook (Member member , Book
    book ) {
    17     if (book. isAvailable()){
    18         book.borrowCopy();
    19         member.borrowBook(book);
    20         System.out.println(member.getName()+"
        borrowed the book : "+book.getTitle());
    21     }
    22     else{
    23         System.out.println("sorry the
        book"+book.getTitle()+"is not available rightnow");
    24     }
    25 }
26 // عملية الإرجاع
27 public void returnBook(Member member , Book book
    ){
    28     if(member.getBorrowedBooks().contains(book)){
    29         book.returnCopy();
    30         member.returnBook(book);
    31         System.out.println(member.getName()+"returned
        the book"+book.getTitle());
    32     }
    33     else{
    34         System.out.println("this member didn't
        borrow this book");
    35     }
    36 }
    37 }

```

■ الخصائص :

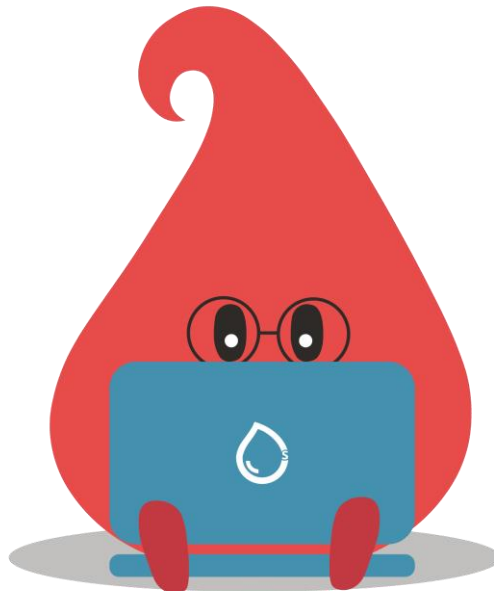
books : قائمة الكتب الموجودة
في المكتبة
members : قائمة الأعضاء
المسجلين

■ الدوال :

addBook() : لإضافة كتاب
addMember() : لإضافة عضو
borrowBook() : لإجراء عملية
الاستعارة
returBook() : لإجراء عملية
الإرجاع



```
1 public class Main {  
2     public static void main(String[] args) {  
3         Library library = new Library();  
4         Book b1 = new Book("Java Basic", "Mohammad", 1, 2);  
5         Book b2 = new Book("OOP", "John", 2, 1);  
6         Member m1 = new Member("Adam", 101);  
7         Member m2 = new Member("Sara", 102);  
8         library.addBook(b1);  
9         library.addBook(b2);  
10        library.addMember(m1);  
11        library.addMember(m2);  
12        library.borrowBook(m1, b1);  
13        library.borrowBook(m2, b2);  
14        library.returnBook(m1, b1);  
15    }  
16 }
```



THE END